## LOVELY PROFESSIONAL UNIVERSITY

School of Computer Science and Engineering

# ACADEMIC TASK–3 PROJECT REPORT

**Course:** CSE316 – Operating Systems **Project:** Real-Time Memory Allocation Tracker

**Student Name:** _____ **Registration No.:** _____

**Section:** _____ **Faculty:** _____ **Date:** _____

**Academic integrity note:** This report is a project draft and learning aid. The assignment instructions require students to code the project themselves. Run, test, understand, adapt and document your own implementation before submitting. Do not claim uncompleted work or GitHub revisions.

## 1. Project Overview

**Abstract.** Real-Time Memory Allocation Tracker is a desktop educational application that demonstrates two memory-management techniques used in operating systems: paging and segmentation. It accepts process identifiers and memory sizes, simulates allocation and deallocation, and refreshes visual memory maps, utilization indicators, mapping tables and event logs after each operation. An optional automatic demonstration generates sample allocation and release events so that changes can be observed without entering every request manually.

**Problem statement.** Memory-management concepts can be difficult to understand when presented only through definitions and static diagrams. Learners need a way to observe how processes occupy memory, how paging maps pages to frames, and how segmentation places variable-sized contiguous blocks. This project addresses that learning need through an interactive simulation.

**Objectives.** (1) Demonstrate page-to-frame mapping. (2) Demonstrate variable-sized contiguous allocation using First-Fit. (3) Allow allocation and deallocation using user inputs. (4) Visualize used and free memory. (5) Display utilization and event history. (6) Provide a repeatable auto-demo and reset option.

**Scope and limitation.** The application models memory using abstract units. Paging uses 24 frames with 4 units per frame; segmentation uses a 100-unit memory space. It is a simulator and does not read, monitor or change actual computer RAM.

## 2. Module-Wise Breakdown

**Module 1 – User Interface and Dashboard:** Built with Tkinter. It provides tabs, process input fields, allocate/deallocate controls, utilization indicators, reset and auto-demo controls. It validates user input and presents feedback.

**Module 2 – Paging Engine:** Divides a process into fixed-size pages and places them in available frames. The engine calculates required pages using the ceiling of process size divided by page size, checks capacity, stores frame mappings and releases frames on deallocation.

**Module 3 – Segmentation Engine and Visualization:** Maintains variable-size contiguous blocks. It uses First-Fit to find the first free hole large enough for a request, tracks free holes and displays the memory layout. It also calculates an external-fragmentation indicator.

## 3. Functionalities

- Allocate memory for a process by entering a process ID and positive size.
- Reject duplicate process IDs within the same memory model.
- In paging, calculate the number of pages and assign free frames.
- Show the page table with process ID, page number and frame number.
- In segmentation, allocate a contiguous block using First-Fit.

- Display allocated blocks and free holes on a memory map.
- Deallocate a process and release its frames or segment.
- Show paging and segmentation utilization, free frames and free memory.
- Record allocation and deallocation events in an event log.
- Run an automatic demonstration and reset the simulation.

## 4. Technology Used

**Programming language:** Python 3.9+.
**GUI library:** Tkinter and ttk (Python standard library).
**Core concepts:** Paging, page tables, frames, segmentation, First-Fit allocation, free holes and external fragmentation.
**Testing:** Python unittest.
**Version control:** Git and GitHub (repository link to be added by the student).

## 5. Flow Diagram

The project flow is: launch application → choose paging or segmentation → enter process ID and size → validate request → check available frames or a suitable contiguous hole → allocate or show an error → refresh visualization and event log → allow deallocation, auto-demo, reset or exit. The editable Mermaid source is supplied as flowchart.mmd in the project folder.

```
flowchart TD
    A([Start]) --> B[Launch Tkinter application]
    B --> C{Choose memory model}
    C -->|Paging| D[Enter process ID and size]
    C -->|Segmentation| E[Enter process ID and size]
    D --> F[Calculate required pages]
    F --> G{Enough free frames?}
    G -->|Yes| H[Assign free frames and update page table]
    G -->|No| I[Show allocation failure]
    E --> J[Find first contiguous free hole]
    J --> K{Hole large enough?}
    K -->|Yes| L[Allocate segment and update memory map]
    K -->|No| M[Show allocation failure]
    H --> N[Refresh dashboard and event log]
    I --> N
    L --> N
    M --> N
    N --> O{Deallocate, auto-demo, reset, or continue?}
    O -->|Deallocate| P[Release frames or segment]
    P --> N
    O -->|Continue| C
    O -->|Reset| Q[Clear simulation state]
    Q --> N
    O -->|Exit| R([End])
```

## 6. Revision Tracking on GitHub

**Repository name:** real-time-memory-allocation-tracker
**GitHub link:** [Add your actual repository URL after creating it]
**Required practice:** Create a repository, commit working changes regularly, use a feature branch for a module, test it, and merge it into main. The assignment requires at least 7 revisions. The following is a suggested sequence; it is not evidence that commits have already been made:

| No. | Suggested commit message |
|---|---|
| 1 | Initial project structure and README |
| 2 | Implement paging allocation engine |
| 3 | Implement segmentation first-fit engine |
| 4 | Build Tkinter dashboard and controls |
| 5 | Add memory visualizations and page table |
| 6 | Add event log, auto demo and reset |
| 7 | Add unit tests and final documentation |

## 7. Conclusion and Future Scope

The tracker provides an interactive way to compare paging and segmentation. Paging demonstrates fixed-size allocation and page-to-frame mapping, while segmentation demonstrates variable-size contiguous allocation and the effect of free-space holes. The dashboard makes allocation changes visible and records operations for review. Future improvements may include configurable memory sizes, Best-Fit and Worst-Fit strategies, FIFO/LRU page replacement demonstrations, process priority, exportable event logs, and more detailed performance statistics.

## 8. References

1. CSE316 Operating Systems Academic Task-3 assignment brief, Lovely Professional University, supplied PDF (especially pages 1–3 for submission format and page 10 for the selected problem statement).
2. Python documentation: https://docs.python.org/3/
3. Tkinter documentation: https://docs.python.org/3/library/tkinter.html
4. Operating Systems course notes/textbook prescribed by the faculty (add the exact title/edition used).

# APPENDIX A – AI-GUIDED PROJECT ELABORATION / BREAKDOWN

**Project overview:** The application simulates memory allocation in two models and provides visual feedback. The intended outcome is a working desktop learning tool for observing allocation, deallocation, utilization and fragmentation-related behavior.

**Module roles:** (1) Tkinter interface collects input and presents results. (2) Paging manager maintains fixed frames and page tables. (3) Segmentation manager maintains variable-size blocks and free holes using First-Fit.

**Example:** With page size 4 units, a process requesting 9 units requires 3 pages and therefore 3 frames. In segmentation, a process requesting 18 units needs one contiguous 18-unit hole; total free memory alone is not sufficient if no single hole is large enough.

**Execution plan:** Install Python; inspect the engine; run unit tests; launch app.py; test successful and failed allocations; test deallocation; use auto-demo; inspect the event log; make changes; commit each working stage to GitHub.

# APPENDIX B – PROBLEM STATEMENT

Real-Time Memory Allocation Tracker
*Description from the supplied CSE316 project list:* “Build a tool that visualizes memory allocation in real-time, showcasing paging and segmentation.” (Assignment PDF, page 10.)

# APPENDIX C – SOLUTION / CODE

The complete source code is included in the accompanying project ZIP: app.py, memory_engine.py and tests/test_memory_engine.py. The source files are provided separately so that they can be executed, tested, read and customized. Do not paste code into the report without first checking that it matches the version you actually developed and tested.

## Student verification checklist

- [ ] I can explain the difference between paging and segmentation.
- [ ] I can explain why a 9-unit process needs 3 pages when page size is 4.
- [ ] I can explain First-Fit and why a large total free space may still fail a contiguous request.
- [ ] I have run the application and tested allocation and deallocation in both tabs.
- [ ] I have run the unit tests and understood any failures.
- [ ] I have created my own GitHub repository and made at least 7 real revisions.
- [ ] I have replaced all placeholders and checked the final report.